Helm Crash Course — Kubernetes Administration

Duration: 45–60 minutes | **Level:** Intermediate | Hands-on. For users familiar with kubectl, Deployments, Services, ConfigMaps, Secrets and rolling updates.

Goal: deploy, configure, upgrade, troubleshoot and roll back applications with Helm. Helm manages application releases; kubectl and Minikube remain important for cluster operations.

1. What is Helm?

Helm is a package manager for Kubernetes. A **chart** packages templates, default values and metadata. Installing a chart creates a **release**, an instance tracked by Helm.

Kubernetes	Helm equivalent
kubectl apply -f	helm install
kubectl get deployments	helm list (releases)
kubectl apply with changes	helm upgrade
kubectl rollout undo	helm rollback
kubectl delete -f	helm uninstall
YAML configuration	values.yaml
Resource YAML definitions	Chart templates

2. Install Helm

On Oracle Linux, the following example installs Helm using the official installer script. Check the installed major version; Helm 3 and Helm 4 have some different flags.

```
curl -fsSL -o get_helm.sh \
  https://raw.githubusercontent.com/helm/helm/main/scripts/get-helm-3
chmod 700 get_helm.sh
./get_helm.sh
helm version
kubectl cluster-info
```

Official documentation: https://helm.sh/docs/intro/install/

3. Lab 1 — Deploy an Application Using Helm

A. Add a repository and search for a chart:

```
helm repo add bitnami https://charts.bitnami.com/bitnami
helm repo update
helm search repo nginx
```

B. Install NGINX:

```
kubectl create namespace demo
helm install mynginx bitnami/nginx --namespace demo
```

Here, mynginx is the release name, bitnami/nginx is the chart, and demo is the namespace. Bitnami image distribution and licensing have changed; verify current image availability before this lab.

C. Verify:

```
helm list -n demo
helm status mynginx -n demo
kubectl get pods,deploy,svc -n demo
```

4. Lab 2 — Change Application Configuration

Inspect chart defaults, then supply a values file instead of editing deployed Kubernetes YAML directly.

```
helm show values bitnami/nginx > default-values.yaml
```

my-values.yaml

```
replicaCount: 3
service:
  type: ClusterIP

helm upgrade mynginx bitnami/nginx -n demo -f my-values.yaml
kubectl get pods -n demo
helm get values mynginx -n demo
```

For a quick override while retaining prior release values:

```
helm upgrade mynginx bitnami/nginx -n demo \
  --reuse-values --set replicaCount=2
```

5. Lab 3 — Upgrades, History and Rollbacks

```
helm history mynginx -n demo
helm upgrade mynginx bitnami/nginx -n demo \
  --reuse-values --set replicaCount=4
helm history mynginx -n demo
helm rollback mynginx 1 -n demo
helm status mynginx -n demo
kubectl get pods -n demo
```

A rollback creates a new revision from earlier configuration. Use --wait and --timeout to wait for readiness. Helm 3 supports --atomic to roll back a failed upgrade; Helm 4 has updated rollback-on-failure options.

```
helm upgrade mynginx bitnami/nginx -n demo \
  -f my-values.yaml --wait --timeout 5m
```

6. Lab 4 — Create Your Own Helm Chart

```
helm create myapp
cd myapp
tree .

myapp/
  Chart.yaml
  values.yaml
  charts/
  templates/
    deployment.yaml
    service.yaml
    ingress.yaml
    serviceaccount.yaml
    _helpers.tpl
    tests/
```

File	Purpose
Chart.yaml	Chart name, version and metadata
values.yaml	Default configuration
templates/deployment.yaml	Kubernetes Deployment template
templates/service.yaml	Kubernetes Service template
templates/_helpers.tpl	Reusable template functions
charts/	Dependent charts

values.yaml:

```
replicaCount: 2
image:
  repository: nginx
  tag: "1.27"
```

Template fragment (not a complete Deployment):

```
spec:
  replicas: {{ .Values.replicaCount }}
  template:
    spec:
      containers:
        - name: nginx
          image: "{{ .Values.image.repository }}:{{ .Values.image.tag }}"
```

Helm substitutes values before submitting manifests to Kubernetes. Validate and preview before installing:

```
helm lint .
helm template myapp .
helm install myapp . -n demo
```

7. Lab 5 — Troubleshooting Helm Deployments

```
helm status myapp -n demo
helm history myapp -n demo
helm get values myapp -n demo
helm get manifest myapp -n demo
kubectl get all -n demo
kubectl get events -n demo --sort-by=.lastTimestamp
kubectl describe pod <pod-name> -n demo
kubectl logs <pod-name> -n demo
```

Helm can install resources before the application is healthy. Use --wait, probes and kubectl events to diagnose failures. For CreateContainerConfigError, inspect missing Secrets or ConfigMaps.

8. Helm Commands Cheat Sheet

Task	Command
Version	helm version
Add repository	helm repo add NAME URL

Task	Command
Update repositories	helm repo update
Search charts	helm search repo nginx
Show defaults	helm show values CHART
Install	helm install APP CHART
List releases	helm list -A
Status	helm status APP -n NS
Upgrade	helm upgrade APP CHART -n NS
Install or upgrade	helm upgrade --install APP CHART -n NS
History	helm history APP -n NS
Rollback	helm rollback APP REV -n NS
Release values	helm get values APP -n NS
Rendered manifests	helm get manifest APP -n NS
Lint chart	helm lint ./myapp
Preview	helm template APP ./myapp
Package	helm package ./myapp
Uninstall	helm uninstall APP -n NS

9. Enterprise DevOps: Helm with CI/CD

Typical pipeline: Git source → CI build and test → container registry → Helm release → Kubernetes workloads.

```
helm upgrade --install orderservice ./helm/orderservice \
  --namespace production \
  --create-namespace \
  --values ./helm/values-prod.yaml \
  --set image.tag="$BUILD_NUMBER" \
  --wait --timeout 5m
```

Use immutable image tags or digests, protect deployment credentials, keep secrets out of plaintext values files, and add environment-specific configuration and rollback procedures.

10. Convert Existing Kubernetes YAML Files into a Helm Chart

Example based on six microservices manifests: api-gateway-deployment.yaml, caching-ms-deployment.yaml, employee-ms-deployment.yaml, frontend-deployment.yaml, postgres-deployment.yaml and redis-deployment.yaml. Each original file may contain multiple resources separated by --- (Deployment, Service, ConfigMap, PVC).

A. Generate a starter chart and copy existing manifests:

```
# From the directory containing the original YAML files:
```

```
helm create java-ms
find java-ms/templates -type f ! -name '_helpers.tpl' -delete
cp *-deployment.yaml java-ms/templates/
```

Remove hardcoded namespace: default from templates so -n java-ms controls the namespace. Preserve service names and selectors used for application-to-application DNS and routing. Do not rename PVCs or Services casually.

B. Chart.yaml:

```
apiVersion: v2
name: java-ms
description: Microservices Helm chart
type: application
version: 0.1.0
appVersion: "1.0.0"
```

C. values.yaml example:

```
apiGateway:
  replicas: 1
  image: nginx:alpine
employeeMs:
  replicas: 3
  image: employee-service:latest
cachingMs:
  replicas: 2
  image: cache-service:latest
frontend:
  replicas: 1
  image: nginx:alpine
postgres:
  replicas: 1
  image: postgres:15-alpine
redis:
  replicas: 1
  image: redis:7-alpine
  storage: 1Gi
```

D. Replace hardcoded fields in the relevant Deployment templates:

```
spec:
  replicas: {{ .Values.employeeMs.replicas }}
# Under spec.template.spec.containers:
  image: {{ .Values.employeeMs.image | quote }}
```

Repeat for each Deployment. Retain the complete manifest structure, selectors, labels, ports, volumes, probes and env vars. Split multi-document YAML files into individual templates if easier to maintain. For long frontend HTML and NGINX configuration, consider placing the content in chart files/ and loading it with .Files.Get, or keep it in ConfigMap templates while protecting any literal {{ sequences.

E. Handle Secrets and persistent data:

```
kubectl create namespace java-ms
kubectl create secret generic postgres-credentials \
  --from-literal=password='REPLACE_WITH_LAB_PASSWORD' \
  -n java-ms
```

Templates can reference this existing Secret with valueFrom.secretKeyRef (name: postgres-credentials, key: password). The Secret must exist in the release namespace before installation. PostgreSQL should use persistent storage for any non-disposable data; Redis PVCs may outlive a release or be deleted depending on how resources and storage policies are configured.

F. Validate and deploy:

```
helm lint ./java-ms
helm template myservices ./java-ms -n java-ms
minikube image ls | grep -E 'employee-service|cache-service'
helm install myservices ./java-ms -n java-ms \
  --wait --timeout 10m
helm status myservices -n java-ms
kubectl get pods,deploy,svc,pvc -n java-ms
```

Your original employee-service and cache-service images use imagePullPolicy: Never; ensure both images are available inside Minikube, e.g. minikube image load employee-service:latest and minikube image load cache-service:latest.

G. Day-to-day management:

```
helm upgrade myservices ./java-ms -n java-ms \
  --set employeeMs.replicas=4 --wait
helm history myservices -n java-ms
helm rollback myservices 1 -n java-ms
kubectl get events -n java-ms --sort-by=.lastTimestamp
```

11. Clean Up Deployments and Reinstall

Option A — Remove just the Helm release:

```
helm uninstall myservices -n java-ms
helm list -n java-ms -a
```

An uninstall removes Helm-managed resources, but externally created Secrets and some persistent storage may remain. Inspect before deleting any data.

Option B — Completely reset the java-ms namespace:

```
# DESTRUCTIVE: deletes resources and PVCs in java-ms.
helm uninstall myservices -n java-ms
kubectl delete namespace java-ms
kubectl create namespace java-ms
kubectl create secret generic postgres-credentials \
  --from-literal=password='REPLACE_WITH_LAB_PASSWORD' \
  -n java-ms
helm install myservices ./java-ms -n java-ms \
  --wait --timeout 10m
```

Namespace deletion may permanently delete persistent data. It does not remove the local Helm chart directory. Verify namespace deletion has completed before recreating it.

Option C — Reset all of Minikube (most destructive):

```
minikube delete
minikube start
# Rebuild or reload locally built images before redeploying.
```

Use Option C only if the entire cluster is disposable. It removes all workloads and cluster-local data, not merely the java-ms application.